

PFE

Primenjena Fizika i Elektronika
Zimski kamp za starije polaznike

Vežba

Postavljanje ćelija i rutiranje

N. Petrović



Mart 2026

Vežve su bazirane na: <https://pulp-platform.org/efclwinter2026/>

1 Pregled

U vežbi posvećenoj floorplanning-u i mreži napajanja upoznali smo se sa podešavanjem *back-end* toka dizajna u OpenROAD-u, dodavanjem pad frame-a, postavljanjem makro blokova i generisanjem mreže napajanja. U ovoj vežbi nastavljamo *back-end* tok dizajna od mesta na kome smo stali i prelazimo na njegove ključne korake: postavljanje i rutiranje. Kroz ovu vežbu naučićete:

- Kako postaviti standardne ćelije
- Kako sintetizovati stablo takta
- Kako izvršiti rutiranje signala
- Kako se uveriti da vaš čip nakon back-end dizajna i dalje ispunjava sva ograničenja tajminga

Polazna tačka je sačuvani checkpoint sa prethodne vežbe.

1.1 O stilu

Zadatak za studente:

Delovi teksta sa sivom pozadinom označavaju korake potrebne za završavanje vežbe.

Tokom vežbe unosite komande putem komandne linije¹.

1.2 Početak rada

Zadatak 1:

- Pokrenite docker

```
sh > ./run_docker.sh
```

- Prebacite se u direktorijum:

```
sh > cd exercises/asic/openroad
```

Napomena 1: Ako otvorite novi terminal, ne zaboravite da ponovo pokrenete docker.

Napomena 2: Celu vežbu radimo u `openroad` folderu.

1.3 Ponovno učitavanje sačuvanih dizajna

U bilo kom trenutku tokom ove ili neke od prethodnih vežbi možete uraditi nešto nenamerno ili želiti da detaljnije istražite neki korak projektovanja. Ponovno pokretanje celog postupka projektovanja oduzima dosta vremena. Da bi izbegli pokretanje celog postupka svaki put kada želite nešto da izmenite ili ispravite je da sačuvate dizajn koji možete kasnije ponovo da učitate:

```
OpenROAD > save_checkpoint <ime>
OpenROAD > load_checkpoint <ime>
```

¹Komandna linija omogućava skriptovanje i ponovljivost.

Napomena 3: Korake ove vežbe je potrebno da upisujete i u OpenROAD terminal i u praznu tcl skriptu koja je pripremljena za vas i nalazi se u `scripts/02_pfe_placement.tcl`. Ne zaboravite, nalazimo se u `openroad` folderu.

2 OpenROAD konfiguracija

U ovoj vežbi potrebno je da unesete nekoliko komandi u OpenROAD terminal. Vodite računa da sačuvate komande u skripti `scripts/02_pfe_placement.tcl` kako bi mogli kasnije da ih ponovo pokrenete. Ako želite da izađete iz terminala i ponovo pokrenete komande, možete ukucati u OpenROAD terminalu:

```
OpenROAD > source scripts/02_pfe_placement.tcl
```

Zadatak 2: Setup

- Pokrenuti OpenROAD gui:

```
sh > openroad -gui
```

- Prvi korak je podešavanje OpenROAD-a. Možete koristiti priloženu skriptu `startup.tcl` da postavite neke opšte pomoćne promenljive i funkcije na koje se druge skripte mogu oslanjati. Pogledajte skriptu da biste videli šta ona radi.

```
OpenROAD > source scripts/startup.tcl
```

Sada kada je alat podešen, potrebno je učitamo šta smo odradili u prethodnoj vežbi:

```
OpenROAD > load_checkpoint 01_pfe.floorplan
```

Sada smo spremni da krenemo sa fazom postavljanja ćelija i rutiranjem.

2.1 Priprema netliste

Pre prvog koraka postavljanja i rutiranja, moramo ‘očistiti’ netlistu. Prvo želimo da dupliramo drajvere za konstantne vrednosti 0 i 1 da ne bismo trošili naše dragoceno vreme na rutiranje ovih vrednosti. Pored ovoga, baferovanje iz alata za sintezu možda nije baš ono što želimo u ovoj fazi projektovanja. Tako da je možda najbolje da i to sada ponovo odradimo.

Zadatak 3: Priprema

- Prvo je potrebno postaviti slojeve metala na osnovu kojih će alat proceniti grube vrednosti parazitnih efekata za postavljanje ćelija:

```
OpenROAD > set_wire_rc -clock -layer Metal3  
OpenROAD > set_wire_rc -signal -layer Metal3
```

- Nakon ovoga, potrebno je da kažemo alatu da ne koristi pad-ove ya rutiranje ili postavljanje jer smo ih već postavili:

```
OpenROAD > set_dont_use_pads [list sg13g2_IOPad* ]  
OpenROAD > set_dont_use $dont_use_pads
```

- Zatim, takodje ne želimo u ovoj fazi da diramo takti signal s obzirom da ćemo imati poseban korak za to kasnije:

```
OpenROAD > set clock_nets \  
    [get_nets -of_objects [get_pins -of_objects "*_reg" -filter "name == CLK"]]  
OpenROAD > set_dont_touch $clock_nets
```

- Sada možemo duplirati drajvere za konstante 0 i 1:

```
OpenROAD > repair_tie_fanout "TIEHI/Y"  
OpenROAD > repair_tie_fanout "TIELO/Y"
```

- Uklonimo sve bafere koji već postoje jer želimo sami da ih generišemo ponovo ukoliko budu bili potrebni:

```
OpenROAD > remove_buffers
```

- I pokrenimo komandu za popravku DRC grešaka i eventualno dodavanje bafera:

```
OpenROAD > repair_design -verbose
```

- Sačuvajmo progres koji smo odradili do sada:

```
save_checkpoint 02-01_pfe.pre_place
```

3 Rasporedjivanje ćelija

Postavljanje ćelija se često deli na dva dela:

1. Globalno rasporedjivanje (engl. *global placement*):

- Posmatra problem na nivou celog čipa i pokušava da obezbedi da svaka ćelija bude približno na dobrom mestu. Dozvoljeno je da u određenoj meri krši pravila rasporedjivanja.

2. Detaljno rasporedjivanje (engl. *detailed placement*):

- Posmatra pojedinačne ćelije i obezbeđuje da se one nalaze na dozvoljenom mestu. Ponekad se ovo dodatno deli na jedan korak koji i dalje aktivno pokušava da optimizuje raspored ćelija i jedan korak koji se bavi isključivo pronalaženjem dozvoljenog (praznog) mesta za postavljanje ćelije.

Pitanje šta neki raspored čini „dobrim” ponekad može biti teško odgovoriti. Kada se posmatra ceo proces projektovanja integrisanog kola, „dobar” raspored je onaj koji minimizuje zauzetu površinu, uz to da dizajn ostane pogodan za rutiranje i da obezbedi što je moguće višu radnu frekvenciju. Međutim, potpuno vrednovanje ovih metrika moguće je tek na samom kraju procesa projektovanja, a ne u njegovoj sredini, gde se sada nalazimo. Zbog toga, sada samo pokušavamo da procenimo navedene metrike bez da zaista znamo finalne vrednosti metrika.

3.1 Procene parazitnih efekata

Procena radne frekvencije (između ostalog) zahteva procene parazitskih efekata žica (uglavnom otpornosti i kapacitivnosti). Postoje četiri „nivoa” procene ovih efekata, pri čemu svaki zahteva sve više informacija i zato se može koristiti tek u kasnijim fazama projektovanja:

- (**Wire load model**): U Liberty datotekama (.lib) uključen je model opterećenja voda. Tačan format može da se razlikuje u zavisnosti od toga koliko detalja proizvođač želi da pruži. Modeli opterećenja voda su naročito korisni kada još nemamo dužine vodova/žica i moramo da ih procenimo. Za to proizvođač PDK-a može analizirati različite prethodne projekte, odrediti raspodelu dužina žica u njima, a zatim iz toga dati „najbolju procenu”. Tipično postoji više okarakterisanih „tačaka” koje predstavljaju dizajne različitih veličina.

Važno! OpenSTA (alat za statičku analizu vremena koji koristi OpenROAD) trenutno nije sposoban da interpretira način na koji je model opterećenja voda zapisan u našem PDK-u, tako da ga ne možemo koristiti.

- **Raspoređivanje (*Placement*)**: Sledeći korak je korišćenje procenjenih dužina izvedenih iz trenutnog rasporeda ćelija, zajedno sa karakterističnim parazitima po jedinici dužine, kako bi se procenili ukupni parazitski efekti neke žice. To se radi pomoću komandi `set_wire_rc` (koja OpenROAD-u zadaje karakteristične parazite) i `estimate_parasitics -placement`, koja zatim formira procene.
- **Globalno rutiranje (*Global Routing*)**: Globalno rutiranje ćemo razmatrati kasnije, ali slično globalnom raspoređivanju, ono predstavlja grubu aproksimaciju rutiranja. Korišćenjem komande `estimate_parasitics -routing` možemo dobiti znatno bolju procenu nego kada koristimo samo podatke o rasporedu ćelija.
- **Ekstrakcija parazita (*Parasitic Extraction*)**: Na samom kraju, kada imate stvarnu geometriju svake žice, možete je koristiti za modelovanje različitih električnih ponašanja i time ponovo dobiti tačnije informacije o parazitskim efektima žica.

3.2 Globalno raspoređjivanje

Zadatak 4: Globalno raspoređjivanje

- Postavite broj procesorskih niti koje su na raspolaganju:

```
OpenROAD > set_thread_count <N>
```

Zamenite <N> odgovarajućim brojem.

- Pokrenite globalno raspoređjivanje sa gustinom od 60%:

```
OpenROAD > global_placement -density 0.60
```

- Generisati izveštaje i sačuvati šta je odradjeno:

```
OpenROAD > report_metrics "02-02_pfe.gpl1"
OpenROAD > report_image "02-02_pfe.gpl1" true true
OpenROAD > save_checkpoint 02-02_pfe.gpl1
```

- Nakon realizacije grubog rasporeda ćelija možemo proceniti parazitne efekte, popraviti raspored i sačuvati progres:

```
OpenROAD > estimate_parasitics -placement
OpenROAD > repair_design -verbose
OpenROAD > repair_timing -setup -verbose
OpenROAD > save_checkpoint 02-02_pfe.gpl1_repaired
```

Šta radi komanda `repair_timing -setup`?

Komanda `global_placement` bez dodatnih parametara cilja homogenu gustinu od 60% (0.6) i ne uzima u obzir nikakva vremenska ograničenja. U nastavku ću objasniti šta globalno raspoređivanje pokušava da uradi „ispod haube” i kako možete da rešite određene probleme, naročito kod većih dizajna. Ovo nije direktno relevantno za ovu vežbu, tako da možete da preskočite sledeći deo teksta.

Metrike

Globalno raspoređivanje (*Global Placement*, GPL) interno koristi dve glavne metrike. Takozvanu dužinu ožičenja po poluobimu (*Half-Perimeter Wire Length*, HPWL) i ciljnu gustinu, odnosno koliko se prelazi preko ciljne gustine, što se naziva *overflow*.

GPL želi da dostigne ciljnu gustinu, odnosno da minimizuje *overflow*, a takođe želi i manje ožičenja, što znači da pokušava da minimizuje HPWL. Ove dve metrike se zatim modeluju kao "sile". Jedna je sila *overflow*-a; ona se primenjuje samo na ćelije u oblastima gde ima previše ćelija i razmiče ih. Druga sila je sila žica; ona privlači međusobno povezane ćelije jednu ka drugoj kako bi žice bile što kraće.

Ciljna gustina

GPL uvek radi sa nekom ciljnom gustinom. Bez nje bismo imali samo sile koje privlače ćelije jednu ka drugoj i one ne bi imale razlog da se rasporede. Uopšteno govoreći, manja ciljna gustina olakšava rutiranje dizajna (više resursa za rutiranje po ćeliji), olakšava kasnija dodavanja ćelija (na primer zbog naknadnih popravki vremenskih karakteristika) i olakšava pronalaženje dobrog legalizovanog rasporeda tokom detaljnog raspoređivanja. Sa druge strane, to čini čip većim, povećava cenu i može pogoršati vremenske karakteristike, jer su vodovi u proseku duži.

Raspoređivanje vođeno vremenskim karakteristikama

Vrlo često je mali broj vodova kritičan u smislu da možda neće zadovoljiti ciljnu frekvenciju (ili druge vremenske zahteve) ako su predugi. Zbog toga je privlačno reći alatu za raspoređivanje da tim vodovima da prioritet. Upravo to radi opcija `timing_driven`. Periodično tokom GPL procesa OpenROAD procenjuje koji vodovi su najvažniji za vremenske karakteristike, a zatim povećava silu ožičenja tako da jače privlači povezane ćelije jednu ka drugoj. To u proseku smanjuje dužinu vodova između kritičnih ćelija.

Raspoređivanje vođeno rutiranjem

Određeni delovi velikih dizajna, kao što su ROM (sintetizovan u veliki broj naizgled nasumičnih logičkih kola) ili *scoreboard* koji prati izdate instrukcije u procesoru, mogu imati mnogo više veza između ćelija nego uobičajeni modul. To znači da im je po ćeliji potrebno više resursa za rutiranje ili, drugačije rečeno, svakoj ćeliji je potrebno više traka za rutiranje na metalnim slojevima iznad nje za prenos signala.

Jednostavan način da se ćelijama obezbedi više traka jeste da se smanji lokalna gustina, tako da postojeće resurse za rutiranje deli sa manjim brojem drugih ćelija.

U OpenROAD-u se to postiže korišćenjem opcije `routability_driven`. Slično kao kod raspoređivanja vođenog vremenskim karakteristikama, GPL periodično ulazi u korak ponovnog ponderisanja, s tim da se ovde ne menja sila ožičenja, već prividna površina ćelije. Dakle, procenjuje se rutiranje i identifikuju se oblasti sa velikim zagušenjem. Izračunava se odnos između iskorišćenih i dostupnih resursa za rutiranje. Na osnovu tog odnosa određuje se odgovarajuće povećanje površine za svaku ćeliju na toj poziciji i ono se primenjuje. Na kraju se, koristeći povećanu površinu ćelije, računa i nova odgovarajuća ciljna gustina za tu lokaciju. *Overflow* se

povećava, što razmiče ćelije.

Zadatak 5: Globalno rasporedjivanje nastavak

- Pokrenuti ponovo globalno rasporedjivanje:

```
OpenROAD > global_placement -density 0.60 \  
                             -routability_driven \  
                             -routability_check_overflow 0.30 \  
                             -timing_driven
```

- Sačuvati metrike i progres:

```
OpenROAD > report_metrics "02-02_pfe.gpl2"  
OpenROAD > report_image "02-02_pfe.gpl2" true true  
OpenROAD > save_checkpoint 02-02_pfe.gpl2
```

4 Detaljno rasporedjivanje

Zadatak 6: Detaljno rasporedjivanje

- Pokrenuti detaljno rasporedjivanje:

```
OpenROAD > detailed_placement
```

Kako su sada postavljene ćelije?

- Pokrenuti komandu za mirror-ovanje ćelija:

```
OpenROAD > optimize_mirroring
```

Ova komanda mirror-uje ćelije kako bi smanjila dužinu žica.

- Sada možemo ponovo proceniti parazitne efekte:

```
OpenROAD > estimate_parasitics -placement
```

- Ukoliko je potrebno, sada možemo ponovo probati da popravimo raspored:

```
# opciono  
OpenROAD > repair_design -verbose  
OpenROAD > repair_timing -setup -verbose
```

- I sačuvati progres:

```
OpenROAD > report_metrics "02_pfe.placed"  
OpenROAD > report_image "02_pfe.placed" true true  
OpenROAD > save_checkpoint 02_pfe.placed
```



5 Sinteza stabla takta

Fan-out mreže označava broj ulaza na koje je povezan dati izlaz. Mreže sa velikim *fan-out*-om (koje pobuđuju stotine ili čak hiljade ulaza) zahtevaju poseban tretman u odnosu na standardne međuveze. Svako sinhrono kolo poseduje najmanje jednu mrežu sa velikim *fan-out*-om a to je mreža taktnog signala. Osnovni problem kod ovakvih mreža jeste velika kapacitivnost opterećenja koju je potrebno pobuditi. Svaki pobuđeni ulaz svojom ulaznom kapacitivnošću doprinosi ukupnoj kapacitivnosti opterećenja, dok međuveze neophodne za distribuciju signala do svih tih ulaza dodatno povećavaju tu kapacitivnost.

Postoje tri važna parametra za takve mreže:

1. *Transition time*

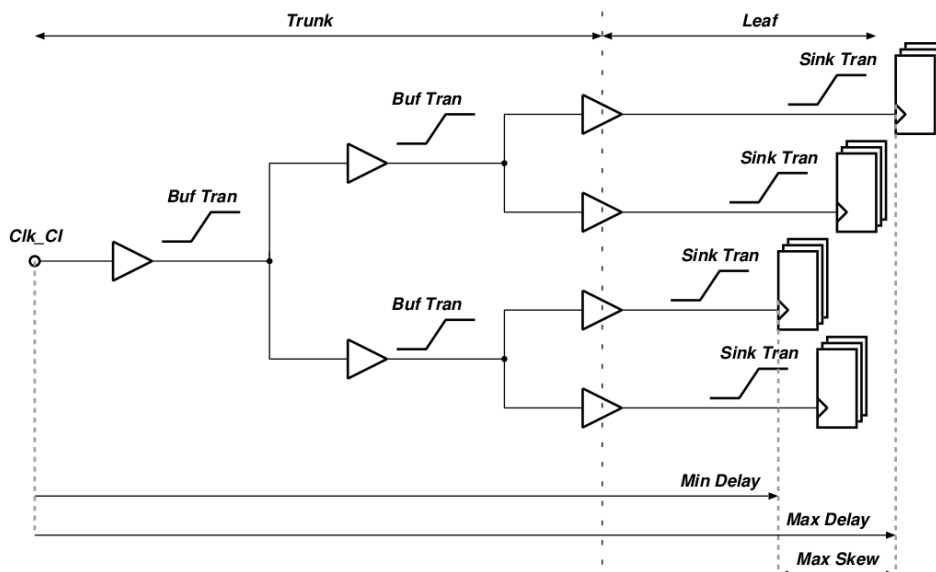
- Predstavlja vreme neophodno za promenu logičkog nivoa na datom čvoru (na primer, $0 \rightarrow 1$). Što je kapacitivno opterećenje na izlazu veće, potrebno je više vremena za njegovo punjenje. CMOS drajveri troše dodatnu struju kratkog spoja tokom prelaza sa 0 na 1 i obrnuto (kada se to dešava?), zbog čega su duga vremena prelaza nepoželjna. Osim toga, šum na signalima sa dugim vremenima prelaza može izazvati pojavu *glitch*-eva. Većina biblioteka definiše gornju granicu za vreme prelaza (za tehnologiju koju koristimo ta granica iznosi 0.56 ns u tipičnim bibliotekama). Radi smanjenja vremena prelaza, moguće je umetnuti stablo bafera kako bi se ukupno opterećenje raspodelilo između njih. Za kraće vreme prelaza potreban je veći broj bafera.

2. *Insertion delay*

- Označava vreme potrebno da signal stigne od drajvera do krajnjih tačaka. Ovo kašnjenje se po pravilu razlikuje za svaku krajnju tačku. Svaki nivo bafera u stablu dodaje određeno kašnjenje na putanji signala.

3. *Skew*

- Predstavlja razliku u kašnjenju umetanja (*insertion delay*) između različitih krajnjih tačaka. Za minimizaciju *skew*-a neophodno je konstruisati uravnoteženo stablo bafera. Generalno, što je zahtevani *skew* manji, to je potreban veći broj bafera.



Danas je naš primarni fokus na signalu takta, pri čemu je ključni zadatak kontrola *skew*-a, s obzirom na to da on direktno utiče na vremenske karakteristike. Maksimalno dozvoljen *skew* određen je periodom taktnog signala. Na primer, pri taktu od 10 MHz, *clock skew* od 1 ns je prihvatljiv. Međutim, pri taktu od 100 MHz, isti *skew* predstavlja 10% periode takta, što bi bilo previše.

Ovaj deo vežbe raditi u skripti `scripts/03_pfe_cts.tcl`.

Zadatak 7: Setup

- Pokrenuti OpenROAD gui:

```
sh > openroad -gui
```

- Prvi korak je podešavanje OpenROAD-a. Koristiti priloženu skriptu `startup.tcl` da postavite neke opšte pomoćne promenljive i funkcije na koje se druge skripte mogu oslanjati. Pogledajte skriptu da biste videli šta ona radi. Ovo je isto kao u prethodnom koraku.

```
OpenROAD > source scripts/startup.tcl
```

Sada kada je alat podešen, potrebno je učitamo šta smo odradili u prethodnoj vežbi:

```
OpenROAD > load_checkpoint 02_pfe.placed
```

Sada smo spremni da krenemo sa fazom CTS projektovanja.

Zadatak 8: CTS

- Prvo je potrebno postaviti slojeve metala na osnovu kojih će alat proceniti grube vrednosti parazitnih efekata:

```
OpenROAD > set_wire_rc -clock -layer Metal3  
OpenROAD > set_wire_rc -signal -layer Metal3
```

- Nakon ovoga, potrebno je da kažemo alatu da ne koristi pad-ove za rutiranje ili postavljanje jer smo ih već postavili:

```
OpenROAD > set_dont_use_pads [list sg13g2_IOPad* ]  
OpenROAD > set_dont_use $dont_use_pads
```

- Zatim je potrebno poništiti komandu za taktne signale koju smo imali u prethodnoj tački (setite se da smo podešavali nešto vezano za taktne signale):

```
OpenROAD > set_clock_nets \  
    [get_nets -of_objects [get_pins -of_objects "*_reg" -filter "name == CLK"]]  
OpenROAD > unset_dont_touch $clock_nets
```

Koja je razlika izmedju ovih komandi i onih koje smo pozivali u prethodnoj tački?



- Dalje je potrebno da pozovemo komandu `repair_clock_inverters`. Ova komanda zamjenjuje inverter u stablu takta koji ima više *fanout*-a sa po jednim inverterom za svaki *fanout*. Ovu komandu treba pokrenuti pre `clock_tree_synthesis`.

```
OpenROAD > repair_clock_inverters
```

- Sada je potrebno da podesimo listu bafera kao i koren CTS stabla (definisano u PDK-u):

```
OpenROAD > set ctsBuf \
    [ list sg13g2_buf_16 sg13g2_buf_8 sg13g2_buf_4 sg13g2_buf_2 ]
OpenROAD > set ctsBufRoot sg13g2_buf_8
```

- Sada možemo da konstruišemo stablo takta:

```
OpenROAD > clock_tree_synthesis -buf_list $ctsBuf \
    -root_buf $ctsBufRoot \
    -sink_clustering_enable \
    -repair_clock_nets
```

- Otvorite GUI (`gui::show`). Istražite generisano stablo takta pomoću GUI alata za prikaz (izaberite **Windows** -> **Clock Tree Viewer** i kliknite na **Update**). Da li generisano stablo takta izgleda kao slika iznad?
- Naš dizajn će raditi na frekvenciji od nekoliko desetina do 100 MHz. Da li *clock skew* deluje razumno u poređenju s tim?
- Koristite komandu za izveštavanje da biste dobili tačnu vrednost (zanimi nas `clk_sys`).

```
OpenROAD > report_clock_skew
```

- Nakon ovoga je dobra praksa legalizovati raspored ćelija:

```
OpenROAD > detailed_placement
```

Dalji koraci služe za sredjivanje CTS-a.

- Procenimo parazitne efekte prvo:

```
OpenROAD > estimate_parasitics -placement
```

- Sada kada imamo CTS, propagirajmo taktni signal:

```
OpenROAD > set_propagated_clock [all_clocks]
```

- Napravimo izveštaj pre sredjivanja:

```
OpenROAD > report_metrics "03_pfe.cts_unrepaired"
```

- Sada možemo da unapredimo CTS (ili makar da probamo):

```
OpenROAD > repair_timing -setup -verbose
```

- Postavimo dodate ćelija (ukoliko su dodate u prethodnom koraku):

```
OpenROAD > detailed_placement
```

- Proverimo da li je sve raspoređeno kako treba

```
OpenROAD > check_placement -verbose
```

Ukoliko je sve kako treba ova komanda treba vratiti 0.

- I na kraju, generisati izveštaje i sačuvati progres:

```
OpenROAD > report_cts -out_file ${report_dir}/03_pfe.cts.rpt  
OpenROAD > report_metrics "03_pfe.cts"  
OpenROAD > report_image "03_pfe.cts" true false true  
OpenROAD > save_checkpoint 03_pfe.cts
```

Ovime je završen CTS deo projektovanja.



CTS done!

6 Rutiranje

Kao i kod rasporedjivanja ćelija, rutiranje je podeljeno na korak globalnog rutiranja i korak detaljnog rutiranja.

Napomena 4: Korake ove vežbe je potrebno da upisujete i u OpenROAD terminal i u praznu tcl skriptu koja je pripremljena za vas i nalazi se u `scripts/03_pfe_routing`. Ne zaboravite, nalazimo se u `openroad` folderu.

Zadatak 9: Setup

Kao i u prethodnom koraku, potrebno je pokrenuti OpenROAD, pokrenuti `startup.tcl` skriptu, učitati prethodni korak i podesiti parametre za estimaciju parazitnih efekata kao:

- Pokrenuti OpenROAD gui:

```
sh > openroad -gui
```

- Pokrenuti skriptu `startup.tcl`:

```
OpenROAD > source scripts/startup.tcl
```

Učitamo šta smo odradili u prethodnoj vežbi:

```
OpenROAD > load_checkpoint 03_pfe.cts
```

- Postaviti slojeve metala na osnovu kojih će alat proceniti vrednosti parazitnih efekata:

```
OpenROAD > set_wire_rc -clock -layer Metal3  
OpenROAD > set_wire_rc -signal -layer Metal3
```

- Nakon ovoga, potrebno je da kažemo alatu da ne koristi pad-ove:

```
OpenROAD > set_dont_use_pads [list sg13g2_IOPad* ]  
OpenROAD > set_dont_use $dont_use_pads
```

Sada smo spremni da krenemo sa fazom rutiranja.

6.1 Globalno rutiranje

Tokom globalnog rutiranja, celokupna površina deli se na ćelije globalnog rutiranja (*GCells*). Svaka ćelija evidentira koliko žica prolazi kroz nju i koliko je vodova dostupno na metalnim slojevima. S obzirom na to da postoje vertikalni i horizontalni slojevi, raspoloživ je određen broj vertikalnih i horizontalnih vodova (engl. *tracks*). Žice se planiraju u strogo horizontalnom/vertikalnom režimu i dodeljuju svim ćelijama kroz koje prolaze. Ukoliko neka *GCell* sadrži više žica nego raspoloživih vodova u bilo kom smeru, neophodno je preusmeriti deo žica tako da prolaze kroz druge *GCell*-ove umesto kroz tu ćeliju.

OpenROAD po pravilu preferira niže slojeve za rutiranje (M2, M3), što je opravdano jer manji broj via-ova ujedno znači i manje izgubljenog prostora i slabije parazitske efekte (razmislite kako izgleda veza između metala u različitim slojevima). Međutim, naročito u završnim fazama detaljnog rutiranja, alat najčešće mora da rešava probleme upravo na nižim metalnim slojevima, budući da svi signali moraju proći kroz njih kako bi se povezali sa ćelijama. Stoga je preporučljivo rezervisati deo prostora na najnižim metalnim slojevima za naknadne korekcije.

Zadatak 10: Globalno rutiranje

- Prvo je potrebno reći alatu koje metale sme da koristi za rutiranje:

```
OpenROAD > set_routing_layers -signal Metal2-TopMetal1 -clock Metal2-TopMetal1
```

U kojim metalima smo dozvolili rutiranje signala?

- Imajte u vidu da su metali TM1 i TM2 značajno veći od ostalih, što ih čini nepogodnim za kratko rutiranje. Ipak, TM1 je neophodan kao horizontalni sloj iznad makro blokova.

Komanda `set_global_routing_layer_adjustment` definiše korekcije resursa za rutiranje po slojevima u okviru dizajna. Ove korekcije smanjuju broj vodova (engl. *tracks*) koje globalni ruter uzima u obzir, čime se podstiče ravnomernija raspodela rutiranja i smanjuje vršna zagušenost, olakšavajući kasniju fazu detaljnog rutiranja.

```
OpenROAD > set_global_routing_layer_adjustment \  
    <StartLayer>-<EndLayer> <reduction>
```

Gde je `<reduction>` broj manji od 1.0 koji određuje udeo blokiranih vodova, a `Start/End-Layer` su nazivi metalnih slojeva; smanjenje važi za njih, kao i za sve slojeve između njih.

- Smanjimo količinu resursa dostupnu za rutiranje metala TM1, pošto tu zapravo ne želimo rutiranje ako je to moguće (a zbog makroa nam ipak treba).

```
OpenROAD > set_global_routing_layer_adjustment TopMetal1 0.20
```

- Pokrenimo sada globalno rutiranje:

```
OpenROAD > global_route \  
    -guide_file ${report_dir}/04_pfe_route.guide \  
    -congestion_report_file ${report_dir}/04_pfe_route_congestionrpt \  
    -allow_congestion
```

Opcija `-allow_congestion` omogućava ruteru da nastavi sa radom čak iako je došlo do zagušenja. Šta rade ostale opcije ove komande?

- Sada možemo da procenimo parazitne efekte nakon globalnog rutiranja i odmah ćemo generisati izveštaje kao i sačuvati progres:

```
OpenROAD > estimate_parasitics -global_routing  
OpenROAD > report_metrics "04-01_pfe.grt"  
OpenROAD > report_image "04-01_pfe.grt" true false false true  
OpenROAD > save_checkpoint 04-01_pfe.grt
```

6.2 Popravke tajminga

Zadatak 11: Popravka tajminga

- U prethodnom koraku smo već procenili parazitne efekte nakon globalnog rutiranja pomoću komande:

```
OpenROAD > estimate_parasitics -global_routing
```

- Pošto će popravke tajminga dodati ćelije, potrebno je da ponovo prilagodimo naše globalno rutiranje kako bi tome odgovaralo; da bismo to uradili, prelazimo u inkrementalni režim.

```
OpenROAD > global_route -start_incremental -allow_congestion
```

- Sada zaista možemo da popravimo dizajn i naš tajming, u ovom slučaju i *setup* i *hold* vremena:

```
OpenROAD > repair_design -verbose
OpenROAD > repair_timing -setup -verbose -repair_tns 100
OpenROAD > repair_timing -hold -hold_margin 0.1 -verbose -repair_tns 100

# Legalizacija rasporeda celija
OpenROAD > detailed_placement
```

- Na kraju, potrebno je da izađemo iz inkrementalnog režima i da pritom izvršimo neophodna prilagođavanja našeg plana rutiranja:

```
OpenROAD > global_route -end_incremental \
    -guide_file ${report_dir}/04_pfe_route.guide \
    -congestion_report_file ${report_dir}/04_pfe_route_congestion.rpt \
    -allow_congestion \
    -verbose
```

- Želimo da otklonimo antenske efekte (veoma duge žice, koji mogu izazvati probleme tokom proizvodnje).

```
OpenROAD > repair_antennas -iterations 5
```

- Sada možemo da sačuvamo progres i generišemo izveštaje:

```
OpenROAD > estimate_parasitics -global_routing
OpenROAD > report_metrics "04-01_pfe.grt_repaired"
OpenROAD > report_image "04-01_pfe.grt_repaired" true true false true
OpenROAD > save_checkpoint 04-01_pfe.grt_repaired
```

6.3 Detaljno rutiranje

Nakon završetka globalnog rutiranja, nalazimo se u poslednjoj fazi *back-end* postupka projektovanja čipova. Preostaje nam "samo" da geometrijski realizujemo plan rutiranja, otklonimo kratke spojeve i slično. To nikako nije jednostavan zadatak i obično oduzima najviše vremena u poređenju sa svim ostalim koracima.

S obzirom na to da ovaj korak traje relativno dugo, pokrenite ga i prepustite računaru da odradi svoj deo posla. U međuvremenu, iskoristite priliku da se protegnete ili napravite kratku pauzu.

Zadatak 12: Detaljno rutiranje

- Podesite broj CPU niti koje su na raspolaganju (što više to bolje):

```
OpenROAD > set_thread_count 8
```

- Pokrenite detaljno rutiranje (mala pauza):

```
OpenROAD > detailed_route -output_drc ${report_dir}/04_pfe_route_drc.rpt \
    -drc_report_iter_step 5 \
    -save_guide_updates \
```

```
-clean_patches \  
-verbose 1
```

DRC izveštaji su veoma korisni za debugovanje potencijalnih problematičnih mesta.

- Sačuvajmo naš progres i izveštaje:

```
OpenROAD > report_metrics "04_pfe.routed"  
OpenROAD > report_image "04_pfe.routed" true false false true  
OpenROAD > save_checkpoint 04_pfe.routed
```

Zadatak 13: Završni detalji

- Ostaje nam još par stvari da odradimo. Kako najverovatnije nismo popunili ceo čip ćelijama, potrebno je da dodamo *filler* ćelije:

```
OpenROAD > set stdfill \  
[ list sg13g2_fill_8 sg13g2_fill_4 sg13g2_fill_2 sg13g2_fill_1 ]  
OpenROAD > filler_placement $stdfill
```

- Zatim pozvati komandu `global_connect` koja se koristi za povezivanje pinova za napajanje i masu na instancama dizajna sa odgovarajućim izvorima napajanja.

```
OpenROAD > global_connect
```

- Generisati finalne izveštaje i sačuvati progres:

```
OpenROAD > report_image "05_pfe.final" true true false true  
OpenROAD > report_metrics "05_pfe.final"  
OpenROAD > save_checkpoint 05_pfe.final
```

- Generisati izlazne fajlove:

```
OpenROAD > write_def out/pfe.def  
OpenROAD > write_verilog -include_pwr_gnd \  
-remove_cells "$stdfill bondpad*" out/pfe_lvs.v  
OpenROAD > write_verilog out/pfe.v  
OpenROAD > write_db out/pfe.odg  
OpenROAD > write_sdc out/pfe.sdc  
  
OpenROAD > define_process_corner -ext_model_index 0 X  
OpenROAD > extract_parasitics -ext_model_file ../ihp13/pdk/ihp-sg13g2/libs.tech  
/librelane/IHP_rcx_patterns.rules  
OpenROAD > write_spef out/pfe.spef
```



Routing done!

7 Sledeći koraci

Čestitamo, završili smo fazu postavljanja i rutiranja. Poslednji deo dizajna biće DRC i LVS provere. Kada naš dizajn prođe te provere, može se proizvesti.

Preporučujemo da što pre prođete kroz tok postavljanja i rutiranja za vaš dizajn. Ovo će vam omogućiti da:

- Pripremite osnovno podešavanje koje možete ponovo koristiti za konačni čip
- Steknete iskustvo i osećaj za dizajn i procenite koliko će vam vremena trebati
- Vidite da li su ograničenja površine i tajminga realna
- Dobijete upotrebljivu procenu potrošnje energije vašeg kola